

التقرير الأكاديمي والتطبيقي الشامل لمشروع التخرج والمقرر

مشروع بناء مترجم لغة البرمجة العربية ومحرر الأكواد المستقل

Arabic Language Compiler (Modern C++20) & Standalone IDE (.NET 9)

اسم المقرر:	بناء المترجمات وتصميم لغات البرمجة (CS 351)
إشراف أستاذ المقرر:	د. خالد الكحصنة
إشراف مدرس العملي:	أ. عقيل الشرفي
أعضاء المجموعة (5 طلاب):	1. يعقوب خالد المهاجري 2. سليمان صالح العربي 3. مالك عادل جبران 4. [اسم عضو المجموعة الرابع] 5. [اسم عضو المجموعة الخامس]
الفصل الدراسي:	الفصل الدراسي الأول
العام الجامعي:	2025 / 2026 م (1446 / 1447 هـ)

1. وصف المشروع وفكرته وهندسته المعمارية

يهدف المشروع إلى تصميم وتطوير نظام برمجي متكامل ومستقل لترجمة وتنفيذ برامج مكتوبة بلغة برمجة عربية عالية المستوى، تتيح للمستخدم كتابة الأكواد باللغة العربية الفصحى مع دعم كامل لاتجاه النصوص (RTL). ينقسم المشروع معماريا إلى برنامجين تنفيذيين مستقلين تماما (Two Independent Executables): المترجم المستقل المكتوب كلياً بلغة Modern C++20 WinForms 9 و C# 13 ومحرر الأكواد المستقل وبيئة التطوير المبنية بلغة (CompilerProject_CPP.exe) (LanguageEditor.exe). من خلال تمرير مسار الملف (Process IPC Pipes) يتم التواصل بينهما عبر مسارات نظام التشغيل القياسية. (JSON) وخيار تبادل البيانات بتنسيق.

يطبق مترجم ++C نموذج خط الأنابيب الكلاسيكي المتكامل ذي المراحل الست المنفصلة (1): (Six-Phase Compiler Pipeline) التحليل المعجمي (Lexical Analysis) لاستخراج الرموز وتطبيع الأرقام ودعم أشكال الهزرات في تعليمات اقرا، (2) التحليل النحوي (Syntax Analysis)، (3) جدول الرموز وإدارة النطاقات والمراجع (AST)، (4) بخوارزمية النزول العودي لبناء شجرة الإعراب المجردة (Semantic Analysis) للتحقق من سلامة الأنواع وحماية الثوابت من التعديل، (5) توليد الشفرة الوسيطة ثلاثية العناوين (6)، (Three-Address Code - TAC) توليد لغة التجميع x86 Assembly بالإضافة إلى مفسر كود وسيط حي لمعالجة مدخلات المستخدم الفورية (TACInterpreter).

2. لغات البرمجة المستخدمة في المشروع

الدور الوظيفي في المشروع	الإصدار المعتمد	لغة البرمجة
تطبيق كافة المراحل الست وهياكل البيانات الشجرية ومفسر الكود الوسيط بأقصى كفاءة بناء نواة المترجم المستقل بالكامل (CompilerProject_CPP.exe)؛ وسرعة.	ISO/IEC 14882:2020 (C++20)	Modern C++
إدارة الملفات، التحرير العربي RTL، محرك التحليل اللحظي، وعرض مخرجات المترجم. بناء بيئة التطوير الرسومية والمحرر المستقل (LanguageEditor.exe)؛	Version 13.0 (.NET 9.0 SDK)	C# (C-Sharp)

3. الأدوات والمكتبات الجاهزة وتوصيفها

الجزء المستخدم فيه	الغرض من الاستخدام	الإصدار	اسم الأداة / المكتبة
كافة مراحل مترجم ++C	الذاكرة الآمنة عبر المؤشرات الذكية وهياكل البيانات الديناميكية وجدول التجزئة السريعة O(1).	C++20	C++ Standard Library (STL)
نواة المترجم ++C	وترجمة كود مترجم ++C إلى ملف تنفيذي أصلي مستقل مع خيارات التحسين القصوى O2.	v19.4x (2022)	Microsoft Visual C++ (MSVC)
ملف main.cpp في ++C	ترميز سطر الأوامر على UTF-8 وقراءة مجرى Stdin للأنابيب الموجهة من المحرر بدون تعليق. ضبط وتوحيد	Win32 API	Windows Console Win32 API
LanguageEditor (MainForm)	واجهة المحرر الرسومية التفاعلية، دعم اتجاه النصوص RTL، وتبويبات عرض المخرجات الثمانية. بناء	v9.0 Core	Windows Forms (.NET 9 Core)
في المحرر MainForm.cs	تشغيل عملية المترجم المستقل في الخلفية وتمرير وسائط JSON ومتابعة قنوات الإدخال والإخراج. إدارة	.NET 9.0	System.Diagnostics.Process
حزمة التثبيت النهائي	ومترجم ++C المستقل معاً لإنتاج ملف إعداد وتثبيت تنفيذي شامل (PREXE-GXX.exe). حزم محرر اللغة	v6.3.3	Inno Setup Compiler

4. الشرح التفصيلي البرمجي لأكواد مترجم ++C وفق العناصر التحليلية التسعة

4.1 نماذج البيانات والرموز الأساسية (Token.h, Node.h, CompilationResult.h)

```
// Token.h & Node.h - نماذج في البيانات مع ++C في الذاكرة إدارة مع
struct Token {
    TokenType Type;
    std::string Value;
    int Line = 0;
    Token(TokenType t, std::string v, int l) : Type(t), Value(v), Line(l) {}
};

struct Node {
    std::string Value; // العقدة نوع (IfStatement, AssignStatement, ReadStatement)
    std::string Name; // بالعقدة المرتبط المعرف اسم
    std::string DataType; // به المصريح البيانات نوع
    std::string Val; // العددية أو النصية القيمة
    int Line = 0; // المصدري السطر رقم
    std::vector<std::shared_ptr<Node>> Children; // الفرعية العقد
    Node(std::string val = "", std::string name = "", int line = 0)
        : Value(val), Name(name), Line(line) {}
};
```

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C وبيئة المحرر
وظيفة الجزء	تعريف الكيانات الأساسية للرموز المعجمية (Token)، وعقد شجرة الإعراب (Node)، وكائن نتيجة الترجمة المتكامل (CompilationResult).
المدخلات (Inputs)	البيانات الأولية المستخلصة من النص المصدري: نوع الرمز، القيمة النصية، ورقم السطر البرمجي.
المخرجات (Outputs)	كائنات مهيكلة بنظام المؤشرات الذكية تمثل الرموز وشجرة الإعراب وقابلة للتسلسل بتنسيق JSON.
المتغيرات المستخدمة	(الأبناء) Children، (رقم السطر) Line، (اسم المتغير) Name، (قيمة الرمز أو تصنيف العقدة) Value، (نوع الرمز المعجمي) Type.
هياكل البيانات المستخدمة	UTF-8 لسلاسل النصوص بترميز std::string لإدارة الشجرة متعددة التفرع، و std::vector<std::shared_ptr<Node>>.
الدوال أو الإجراءات	منشآت الكائنات Node()، Token()، ودالة Node::Print() للطباعة الشجرية، ودالة CompilationResult::ToJson().
المكتبات المستخدمة	المكتبة القياسية [string]، [vector]، [memory]، [sstream]، [C++20].
الملفات المرتبطة	Token.h، TokenType.h، Node.h، CompilationResult.h، المترجم اللاحقة.
الارتباط بباقي أجزاء المشروع	تشكل هذه النماذج العقد البياني الأساسي (Data Contract) الموحد الذي تتداوله كافة مكونات المترجم وتستلمه واجهة المحرر.

4.2 المحلل المعجمي واللغوي (Lexical Analysis – Lexer.h & Lexer.cpp)

```
// Lexer.cpp - في المشرقية الأرقام وتطبيع الهمزات ودعم المعجمي التحليل
const std::unordered_set<std::string> Lexer::Keywords = {
    "رمزي_خط", "حرفي", "منطقي", "حقيقي", "صحيح", "متغير", "نوع", "ثابت", "برنامج",
    "اضف", "الى", "كرر", "حتى", "اعد", "استمر", "طالما", "والا", "فان", "اذا",
    "خطا", "صح", "خطأ", "صواب", "اطيع", "اقرء", "اقرأ", "اقرأ"
};

std::vector<Token> Lexer::Tokenize() {
    std::vector<Token> tokens;
    while (_pos < _src.size()) {
        SkipWhitespaceAndComments();
        if (_pos >= _src.size()) break;
        char c = _src[_pos];
        if (IsArabicLetter(c) || isalpha((unsigned char)c) || c == '_' ) {
            tokens.push_back(ReadIdentifierOrKeyword());
        } else if (isdigit((unsigned char)c) || IsEasternArabicDigit(PeekUtf8(2))) {
            tokens.push_back(ReadNumber());
        } else if (c == '"') {
            tokens.push_back(ReadString());
        } else {
            tokens.push_back(ReadOperatorOrDelimiter());
        }
    }
    tokens.push_back(Token(TokenType::EndOfFile, "الملف_نهاية", _line));
    return tokens;
}
```

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C وبيئة المحرر
وظيفة الجزء	قراءة النص المصدري العربي بترميز UTF-8 وتفكيكه إلى وحدات معجمية، وتطبيع الأرقام المشرقية، ودعم الكلمات المفتاحية بكافة أشكال الهمزات.
المدخلات (Inputs)	الكود المصدري العربي كنص كامل (const std::string& sourceCode).
المخرجات (Outputs)	مصفوفة متتابعة من الرموز المعجمية المرتبة (std::vector[Token]).
المتغيرات المستخدمة	المؤشر الحالي pos_، عداد الأسطر line_، النص المصدري src_، ومصفوفة الرموز المستخرجة tokens.
هياكل البيانات المستخدمة	جدول تجزئة سريع std::unordered_set<std::string> للكلمات المفتاحية، ومصفوفة std::vector[Token].
الدوال أو الإجراءات	Tokenize(), ReadIdentifierOrKeyword(), ReadNumber(), ReadString(), SkipWhitespaceAndComments(), IsEasternArabicDigit().
المكتبات المستخدمة	المكتبة القياسية [C++20: [unordered_set], [vector], [string], [ctype], [sstream].
الملفات المرتبطة	Lexer.h, Lexer.cpp, Token.h, TokenType.h.
الارتباط بباقي أجزاء المشروع	المرحلة الأولى في خط أنابيب المترجم؛ يمرر قائمته المعجمية إلى المحلل النحوي Parser.cpp لبدء التحقق الهيكلي.

4.3 المحلل النحوي وبناء شجرة الإعراب (Syntax Analysis – Parser.h & Parser.cpp)

```
// Parser.cpp - المجردة الإعراب شجرة وبناء LL(1) العودي النزول خوارزمية تطبيق
std::shared_ptr<Node> Parser::ParseReadStatement() {
    int line = Current().Line;
    Advance(); // اقرأ / اقرأ / اقرأ المفتاحية الكلمة تجاوز //
    bool hasParen = Match("("); // الاختياري القوس فحص
    auto varToken = ExpectType(TokenType::Identifier, "إليه القراءة المراد المتغير اسم كتابة يجب");
    if (hasParen) Expect(")", "اقترا لتعليمة ' القوس إغلاق يجب");
    Expect(";", "منقوطة بفاصلة 'اقترا لتعليمة إنهاء يجب");
    auto readNode = std::make_shared<Node>("ReadStatement", varToken.Value, line);
    return readNode;
}

std::shared_ptr<Node> Parser::ParseProgram() {
    Expect("برنامج", "بكلمة الكود يبدأ أن يجب", "برنامج");
    auto progToken = ExpectType(TokenType::Identifier, "البرنامج اسم تحديد يجب");
    Expect(";", "البرنامج اسم بعد منقوطة فاصلة وضع يجب");
    auto root = std::make_shared<Node>("برنامج", progToken.Value, progToken.Line);
    root->Children.push_back(ParseDeclarations());
    Expect("{", "البرنامج كتلة لبدء ' المعقوف القوس فتح يجب");
    root->Children.push_back(ParseStatements());
    Expect("}", "البرنامج لنهاية ' المعقوف القوس إغلاق يجب");
    Expect(".", "بنقطة البرنامج إنهاء يجب");
    return root;
}
```

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C وبيئة المحرر
وظيفة الجزء	التحقق من صحة القواعد النحوية وفق صيغة EBNF وتطبيق خوارزمية LL(1) التنبؤية وبناء شجرة الإعراب المجردة والتعافي من الأخطاء.
المدخلات (Inputs)	مصفوفة الرموز المعجمية الناتجة من مرحلة التحليل المعجمي ([std::vector<Token>]).
المخرجات (Outputs)	مؤشر ذكي للعقدة الجذرية لشجرة الإعراب (std::shared_ptr<Node>) وقائمة الأخطاء النحوية (Errors).
المتغيرات المستخدمة	مؤشر موقع الرمز pos_، مصفوفة الرموز tokens_، وسجل الأخطاء النحوية errors_.
هياكل البيانات المستخدمة	شجرة ديناميكية متعددة الفروع عبر std::shared_ptr<Node>، ومصفوفة سلاسل أخطاء [std::vector<std::string>].
الدوال أو الإجراءات	ParseProgram(), ParseDeclarations(), ParseStatements(), ParseReadStatement(), ParseIfStatement(), Match(), Expect(), Synchronize().
المكتبات المستخدمة	المكتبة القياسية [C++20: [memory], [vector], [string], [iostream]].
الملفات المرتبطة	Parser.h, Parser.cpp, Node.h, Token.h, TokenType.h.
الارتباط بباقي أجزاء المشروع	يمثل الركنية الهيكلية للمترجم؛ حيث يسلم شجرة الإعراب للمحلل الدلالي ثم لمولد الكود الوسيط.

4.4 جدول الرموز وإدارة النطاقات والمراجع (SymbolTable.h & SymbolTable.cpp)

// SymbolTable.cpp - إدارة C++20 في والاستخدام التعريف أسطر وتتبع والثوابت المعرفات إدارة -

```
bool SymbolTable::AddSymbol(const std::string& name, const std::string& type,
                           const std::string& kind, int line, const std::string& val) {
    if (_symbols.find(name) != _symbols.end()) {
        return false; // النطاق نفس في مسبقا معرف الرمز خطأ:
    }
    SymbolInfo info;
    info.Name = name; info.Type = type; info.Kind = kind;
    info.DefinedLine = line; info.Value = val;
    _symbols[name] = info;
    return true;
}

void SymbolTable::AddReference(const std::string& name, int line) {
    auto it = _symbols.find(name);
    if (it != _symbols.end()) {
        auto& lines = it->second.ReferencedLines;
        if (std::find(lines.begin(), lines.end(), line) == lines.end()) {
            lines.push_back(line);
        }
    }
}
```

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم C++ وبيئة المحرر
وظيفة الجزء	تسجيل وحفظ كافة المعرفات في البرنامج (متغيرات، ثوابت، أنواع) وتتبع خصائصها، سطر تعريفها، وأسطر استخدامها، ومنع إعادة التعريف.
المدخلات (Inputs)	بيانات التصريح والاستخدام: اسم المعرف، نوع البيانات، التصنيف (متغير/ثابت)، رقم السطر، والقيمة الأولية.
المخرجات (Outputs)	سجل معلومات الرمز (SymbolInfo)، ونتائج التحقق والاسترجاع بزمن ثابت $O(1)$.
المتغيرات المستخدمة	جدول المعرفات <code>_symbols</code> ، ومعلومات الرمز <code>info</code> ، ومصفوفة أسطر المراجع <code>ReferencedLines</code> .
هياكل البيانات المستخدمة	جدول تجزئة سريع <code>[std::unordered_map[std::string, SymbolInfo]</code> وهيك <code>SymbolInfo</code> مخصص.
الدوال أو الإجراءات	<code>AddSymbol()</code> , <code>Lookup()</code> , <code>Contains()</code> , <code>AddReference()</code> , <code>IsConstant()</code> , <code>GetAllSymbols()</code> , <code>Clear()</code> .
المكتبات المستخدمة	المكتبة القياسية <code>[C++20: [unordered_map], [vector], [string], [algorithm], [iomanip]</code> .
الملفات المرتبطة	<code>SymbolTable.h</code> , <code>SymbolTable.cpp</code> , <code>SemanticAnalyzer.h</code> , <code>CompilerRunner.h</code> .
الارتباط بباقي أجزاء المشروع	المستودع البياني المركزي الذي يستند عليه المحلل الدلالي للتحقق من سلامة الأكواد ومولد الكود لحجز المتغيرات.

4.5 المحلل الدلالي والتحقق من الأنواع وحماية الثوابت (SemanticAnalyzer.h & SemanticAnalyzer.cpp)

```
// SemanticAnalyzer.cpp - الثوابت وحماية الأنواع ومطابقة الاستخدام سلامة فحص
void SemanticAnalyzer::CheckStatement(const std::shared_ptr<Node>& node) {
    if (node->Value == "AssignStatement") {
        auto sym = _symbolTable.Lookup(node->Name);
        if (!sym) {
            AddError("دلالي خطأ: به مصرح غير متغير استخدام: دلالي خطأ", node->Line);
        } else if (sym->Kind == "ثابت") {
            AddError("دلالي خطأ: لا يمكن التعديل على الثابت على الثابت", node->Line);
        }
        _symbolTable.AddReference(node->Name, node->Line);
    } else if (node->Value == "ReadStatement") {
        auto sym = _symbolTable.Lookup(node->Name);
        if (!sym) {
            AddError("دلالي خطأ: دلالي خطأ: محاولة القراءة إلى معرف غير متغير إلى القراءة محاولة: دلالي خطأ", node->Line);
        } else if (sym->Kind == "ثابت") {
            AddError("دلالي خطأ: لا يمكن القراءة إلى الثابت إلى الثابت", node->Line);
        }
        _symbolTable.AddReference(node->Name, node->Line);
    }
}
```

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C وبيئة المحرر
وظيفة الجزء	التحقق من المعنى الدلالي والمنطقي للبرنامج، ومنع استخدام المعارف غير المصرح بها، ومنع الكتابة أو القراءة إلى الثوابت، ومطابقة الأنواع.
المدخلات (Inputs)	شجرة الإعراب المجردة (AST Node) وجدول الرموز (SymbolTable).
المخرجات (Outputs)	قيمة منطقية توضح سلامة البرنامج، وقائمة الأخطاء الدلالية المكتشفة وتحديث أسطر المراجع في جدول الرموز.
المتغيرات المستخدمة	سجل الأخطاء الدلالية errors_، مرجع جدول الرموز symbolTable_، والمعارف المفحوصة sym.
هياكل البيانات المستخدمة	مصفوفة نصوص للأخطاء std::vector<std::string>، ومصفوفة الرموز المرجعية.
الدوال أو الإجراءات	Analyze(), ValidateDeclarations(), CheckStatements(), CheckStatement(), EvaluateExprType(), AreTypesCompatible().
المكتبات المستخدمة	المكتبة القياسية [C++20: [vector], [string], [memory], [sstream].
الملفات المرتبطة	SemanticAnalyzer.h, SemanticAnalyzer.cpp, Node.h, SymbolTable.h.
الارتباط بباقي أجزاء المشروع	يشكل صمام الأمان الفاصل بين الواجهة الأمامية للمترجم ومراحل الواجهة الخلفية؛ إذ يوقف الترجمة فوراً عند وجود خلل دلالي.

4.6 توليد الشفرة الوسيطة ثلاثية العناوين (& IntermediateCodeGenerator.h) IntermediateCodeGenerator.cpp

```
// IntermediateCodeGenerator.cpp - العتاد عن المستقلة TAC تعليمات إنتاج -
void IntermediateCodeGenerator::GenerateStatement(const std::shared_ptr<Node>& node) {
    if (node->Value == "IfStatement") {
        std::string cond = GenerateExpression(node->Children[0]);
        std::string trueLbl = NewLabel(), falseLbl = NewLabel();
        _tac.push_back("if " + cond + " goto " + trueLbl);
        _tac.push_back("goto " + falseLbl);
        _tac.push_back(trueLbl + ";;");
        GenerateStatement(node->Children[1]);
        _tac.push_back(falseLbl + ";;");
    } else if (node->Value == "ReadStatement") {
        _tac.push_back("read " + node->Name);
    } else if (node->Value == "PrintStatement") {
        for (auto& child : node->Children) {
            _tac.push_back("print " + GenerateExpression(child));
        }
    }
}
```

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C وبيئة المحرر
وظيفة الجزء	تحويل شجرة الإعراب الهرمية إلى شفرة خطية ثلاثية العناوين مستقلة عن المعمارية تستخدم متغيرات مؤقتة وسوم قفز شرطي.
المدخلات (Inputs)	شجرة الإعراب المجردة الخالية من الأخطاء الدلالية ([std::shared_ptr<Node>]).
المخرجات (Outputs)	مصفوفة أسطر الكود الوسيط ثلاثي العناوين (std::vector<std::string> TAC).
المتغيرات المستخدمة	عداد المتغيرات المؤقتة (t1, t2_ tempCount)، عداد وسوم القفز (L1, L2_ labelCount)، ومصفوفة التعليمات tac_.
هياكل البيانات المستخدمة	مصفوفة خطية متتابعة std::vector<std::string> لتسجيل التعليمات.
الدوال أو الإجراءات	Generate(), GenerateStatement(), GenerateExpression(), NewTemp(), NewLabel(), GetInstructions().
المكتبات المستخدمة	المكتبة القياسية [string], [vector], [memory], [sstream]. [C++20:]
الملفات المرتبطة	IntermediateCodeGenerator.h, IntermediateCodeGenerator.cpp, Node.h, AssemblyCodeGenerator.h, TACInterpreter.h.
الارتباط بباقي أجزاء المشروع	المرحلة الأولى في الواجهة الخلفية؛ يسلم تعليماته لمفسر الذاكرة الحية TACInterpreter ولمولد لغة التجميع x86.

4.7 توليد لغة التجميع x86 ومفسر الكود الوسيط والتنفيذ (& AssemblyCodeGenerator & TACInterpreter)

```
// TACInterpreter.cpp - مفسر الكود الوسيط وتنفيذ تعليمات في الذاكرة
std::string TACInterpreter::Execute(const std::vector<std::string>& instructions,
                                   const std::string& inputBuffer) {
    std::stringstream out;
    PrepareInputTokens(inputBuffer);
    size_t pc = 0;
    while (pc < instructions.size()) {
        const auto& line = instructions[pc];
        if (line.rfind("read ", 0) == 0) {
            std::string dest = line.substr(5); Trim(dest);
            double val = 0.0;
            if (_inputIndex < _inputTokens.size()) {
                val = stod(_inputTokens[_inputIndex++]);
            }
            _variables[dest] = val;
        } else if (line.rfind("print ", 0) == 0) {
            out << EvaluatePrint(line.substr(6)) << "\n";
        }
        pc++;
    }
    return out.str();
}
```

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C وبيئة المحرر
وظيفة الجزء	توليد كود لغة التجميع x86 بنمط MASM، ومحاكاة وتنفيذ تعليمات TAC في الذاكرة الحية وقراءة مدخلات المستخدم لتعليمات اقرا فوراً.
المدخلات (Inputs)	تعليمات الكود الوسيط TAC، ومدخلات المستخدم الموجهة عبر Stdin.
المخرجات (Outputs)	كود لغة التجميع (.asm)، ومخرجات الكونسول النصية الناتجة عن التشغيل الفعلي.
المتغيرات المستخدمة	عداد البرنامج pc، جدول متغيرات الذاكرة _variables، ومجرى المدخلات _inputTokens.
هياكل البيانات المستخدمة	جدول تجزئة قيم المتغيرات double [std::unordered_map[std::string, double]]، وجدول الوسوم.
الدوال أو الإجراءات	GenerateX86Assembly(), TACInterpreter::Execute(), EvaluatePrint(), PrepareInputTokens(), Trim().
المكتبات المستخدمة	المكتبة القياسية [C++20: [iostream], [sstream], [unordered_map], [string], [cmath]].
الملفات المرتبطة	AssemblyCodeGenerator.h/cpp, TACInterpreter.h/cpp, main.cpp.
الارتباط بباقي أجزاء المشروع	المرحلة الختامية في مترجم ++C تنتج النتائج النهائية للترجمة والتنفيذ وتطبعها بصيغة JSON لتبويب الكونسول في المحرر.

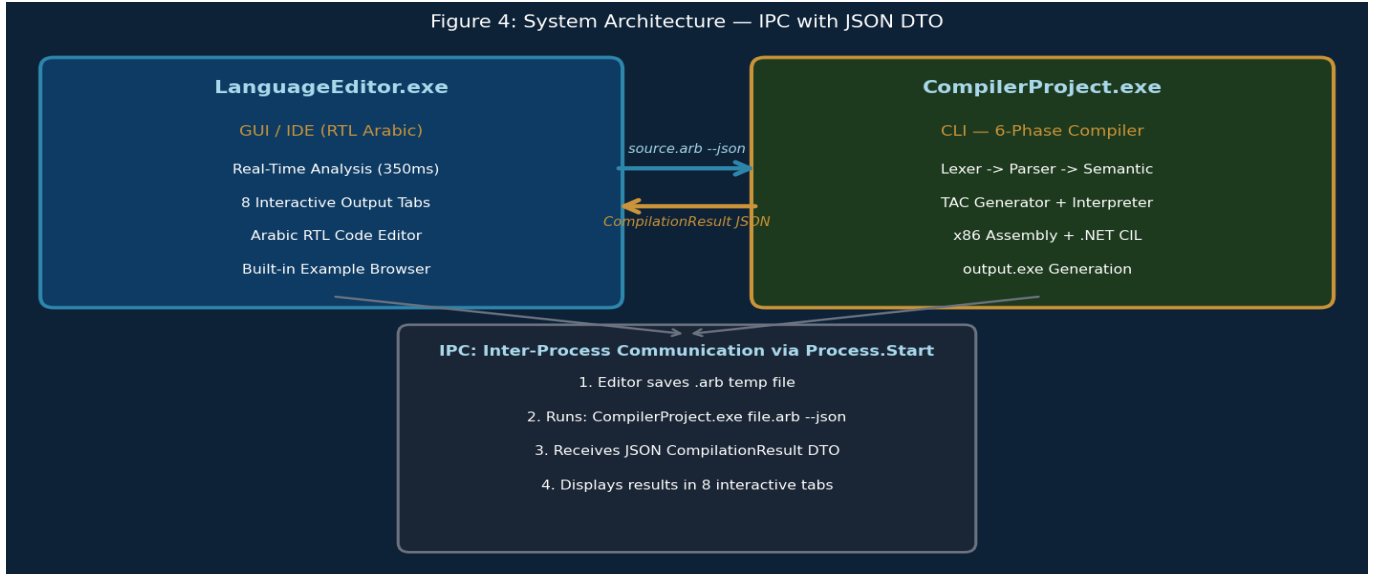
4.8 محرر لغة البرمجة العربية وبيئة التطوير المستقلة (LanguageEditor – MainForm.cs)

```
// MainForm.cs - البرمجة الأنابيب عبر المترجم واستدعاء الرسومية التطوير بيئة -
private async Task RunCompilerEngineAsync(string sourceCode, string userInput) {
    string tempSrc = Path.Combine(Path.GetTempPath(), "arabic_source.arb");
    File.WriteAllText(tempSrc, sourceCode, Encoding.UTF8);
    var psi = new ProcessStartInfo {
        FileName = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "CompilerProject_CPP.exe"),
        Arguments = $"\"{tempSrc}\" --json",
        RedirectStandardInput = true, RedirectStandardOutput = true, UseShellExecute = false,
        StandardOutputEncoding = Encoding.UTF8, CreateNoWindow = true
    };
    using var proc = Process.Start(psi);
    if (!string.IsNullOrEmpty(userInput)) await proc.StandardInput.WriteLineAsync(userInput);
    proc.StandardInput.Close();
    string jsonResult = await proc.StandardOutput.ReadToEndAsync();
    await proc.WaitForExitAsync();
    UpdateInterfaceComponents(jsonResult);
}
```

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C وبيئة المحرر
وظيفة الجزء	توفير بيئة تطوير تفاعلية متكاملة باللغة العربية (RTL)، إدارة الملفات، محرك التحليل اللحظي، حقل مدخلات تعليمية اقرا، وتبويبات النتائج الثمانية.
المدخلات (Inputs)	أكواد المستخدم المكتوبة، تفاعلات الفأرة والمفاتيح، وقيم المدخلات النصية لتعليمية اقرا.
المخرجات (Outputs)	عرض مباشر لنتائج كافة مراحل الترجمة (Tokens, AST, Symbol Table, TAC, Assembly, Console, Errors).
المتغيرات المستخدمة	المؤقت اللحظي liveTimer_، حقل الإدخال inputTextBox_، محرر الأكواد codeEditor_، والتبويبات tabResults_.
هياكل البيانات المستخدمة	عناصر تحكم TreeView للشجرة، DataGridView للجداول، RichTextBox للكونسول، وقوائم الكائنات.
الدوال أو الإجراءات	RunCompilerEngineAsync(), InitializeLiveEngine(), UpdateInterfaceComponents(), BtnRefresh_Click(), LineNumbering_Paint().
المكتبات المستخدمة	.NET 9 Core: System.Windows.Forms, System.Diagnostics, System.Text.Json, System.IO.
الملفات المرتبطة	MainForm.cs, MainForm.Designer.cs, Program.cs, LanguageEditor.csproj.
الارتباط بباقي أجزاء المشروع	الحاضنة الرسومية الشاملة؛ تستدعي مترجم ++C المستقل كعملية منفصلة وتعرض نتائجه للمستخدم بدون أي تجميد للواجهة.

5. الأطر المعمارية والنظرية وقواعد اللغة العربية (EBNF)

Figure 4: System Architecture — IPC with JSON DTO



الشكل (1): معمارية النظام المتكاملة والفصل التام بين محرر اللغة ومترجم ++C المستقل عبر IPC Pipes

قواعد لغة البرمجة بصيغة EBNF الموسعة المعتمدة للمشروع:

```

<برنامج> ::= "برنامج" <معرف> "؛" <كتلة> "."
<كتلة> ::= [ <قسم_التصريحات> ] "}" <قائمة_الجمل> "{"
<قسم_التصريحات> ::= { <تصريح_ثابت> | <تصريح_متغير> | <تصريح_نوع> }
<تصريح_متغير> ::= "متغير" <معرف> "؛" <معرف> "؛" <نوع_البيانات> "؛"
<تصريح_ثابت> ::= "ثابت" <معرف> "=" <قيمة_ثابتة> "؛"
<نوع_البيانات> ::= "صحيح" | "حقيقي" | "منطقي" | "حرفي" | "خيط_رمزي"
<جملة_شرطية> ::= "إذا" "(" <تعبير> ")" "فإن" <جملة> "وإلا" <جملة>
<جملة_كرّر> ::= "كرّر" "(" <معرف> "=" <تعبير> "إلى" <تعبير> [ <اضف> <تعبير> "]" "(" <جملة>
<جملة_طالما> ::= "طالما" "(" <تعبير> ")" "استمر" <جملة>
<جملة_اعد> ::= "اعد" <جملة> "حتى" "(" <تعبير> ")" "؛"
<تعليمية_قراءة> ::= ("اقرأ" | "اقرأ" | "اقرأ") "(" <معرف> ")" "؛"
<تعليمية_طباعة> ::= "اطبع" "(" <تعبير> ")" <تعبير> "{" <تعبير> "}" "؛"
    
```

هرم أسبقية العمليات وتراكيب حلقات التكرار البرمجية

الشكل (2): هرم أسبقية المعاملات الحسابية والمنطقية



الادنى أسبقية ▼

الشكل (2): هرم أسبقية المعاملات الحسابية والمنطقية المعتمدة في المترجم

كرر (حلقة العداد)



طالما (حلقة شرطية)



اعد.. حتى (حلقة لاحقة)



الشكل (3): مخطط تدفق تراكيب حلقات التكرار الثلاث (كرر، طالما، اعد.. حتى)

الشكل (3): مخطط تدفق تراكيب حلقات التكرار الثلاث (كرر، طالما، اعد.. حتى)

6. حزمة الأمثلة الاختبارية العشرة المعتمدة (Test Programs 10)

المثال 1: تعريف المتغيرات والعمليات الحسابية الأساسية

برنامج حساب_المستطيل ؛

متغير

الطول , العرض , المساحة : صحيح ؛

}

الطول = 15 ؛

العرض = 10 ؛

المساحة = الطول * العرض ؛

اطبع ("مساحة المستطيل تساوي:" , المساحة) ؛

.{

الهدف: إثبات صحة التصريح عن المتغيرات وإجراء عمليات الضرب والإسناد والطباعة المركبة.

المخرجات المتوقعة: مساحة المستطيل تساوي: 150

المثال 2: العمليات الحسابية المعقدة وأسبقيات المعاملات

برنامج اسبقية_العمليات ؛

متغير

س , ص , ع , نتيجة : صحيح ؛

}

س = 20 ؛ ص = 4 ؛ ع = 2 ؛

نتيجة = س + ص * ع - 10 / 2 ؛

اطبع ("النتيجة النهائية هي:" , نتيجة) ؛

.{

الهدف: إثبات تطبيق أسبقية الضرب والقسمة قبل الجمع والطرح بدقة.

المخرجات المتوقعة: النتيجة النهائية هي: 23

المثال 3: الجمل الشرطية البسيطة والمركبة (إذا / فان / والا)

برنامج فحص_الدرجات ؛

متغير

الدرجة : صحيح ؛

}

الدرجة = 85 ؛

إذا (الدرجة <= 90) فان {

اطبع ("التقدير: ممتاز") ؛

{ والا }

إذا (الدرجة <= 75) فان {

اطبع ("التقدير: جيد جدا") ؛

{ والا }

اطبع ("التقدير: مقبول") ؛

{ ؛

{ ؛

. {

الهدف: إثبات إعراب الشروط المتداخلة والقفزات الشرطية في TAC.

المخرجات المتوقعة: التقدير: جيد جدا

المثال 4: حلقة العداد التراكمية المحددة (كرر)

برنامج مجموع_الاعداد ؛

متغير

العداد , المجموع : صحيح ؛

}

المجموع = 0 ؛

كرر العداد من 1 الى 10 اضع 1 {

المجموع = المجموع + العداد ؛

{ ؛

اطبع ("مجموع الأرقام من 1 إلى 10 هو:" , المجموع) ؛

. {

الهدف: إثبات بناء عقد حلقة كرر وتوليد قفزات الزيادة التلقائية للعداد.

المخرجات المتوقعة: مجموع الأرقام من 1 إلى 10 هو: 55

المثال 5: حلقة الدوران بالشرط المسبق (طالما ... استمر)

برنامج حساب_القوى ؛

متغير

الاساس , العداد , الناتج : صحيح ؛

}

الاساس = 2 ؛ العداد = 1 ؛ الناتج = 1 ؛

طالما (العداد >= 5) استمر {

الناتج = الناتج * الاساس ؛

العداد = العداد + 1 ؛

؛ {

اطبع ("2 أس 5 يساوي:" , الناتج) ؛

. {

الهدف: إثبات فحص الشرط المسبق وتكرار الكتلة البرمجية.

المخرجات المتوقعة: 2 أس 5 يساوي: 32

المثال 6: حلقة الدوران بالتحقق اللاحق (اعد ... حتى)

برنامج اختبار_اعد_حتى ؛

متغير

قيمة_س : صحيح ؛

}

قيمة_س = 1 ؛

اعد {

قيمة_س = قيمة_س * 2 ؛

{ حتى (قيمة_س <= 16) ؛

اطبع ("القيمة بعد انتهاء الحلقة هي:" , قيمة_س) ؛

. {

الهدف: إثبات تنفيذ جسم الحلقة مرة واحدة على الأقل وفحص شرط الخروج اللاحق.

المخرجات المتوقعة: القيمة بعد انتهاء الحلقة هي: 16

المثال 7: تعلية القراءة من المستخدم (اقرا) والطباعة

برنامج مضاعفة المدخلات ؛

متغير

العدد_المدخل , الناتج : صحيح ؛

}

اطبع ("ادخل عددا ليتم مضاعفته:") ؛

اقرا (العدد_المدخل) ؛

الناتج = العدد_المدخل * 2 ؛

اطبع ("ضعف العدد المدخل هو:" , الناتج) ؛

{ .

الهدف: إثبات معالجة مدخلات المستخدم من حقل الإدخال عبر Stdin بدون تعليق المترجم.

المدخل التجريبي: 25 | المخرجات: ضعف العدد المدخل هو: 50

المثال 8: البرنامج الشامل لكافة القواعد والتراكيب

برنامج النظام_الشامل ؛

ثابت

النسبة_المئوية = 100 ؛

متغير

الرقم , المجموع , الضرب : صحيح ؛

}

الرقم = 5 ؛ المجموع = 0 ؛ الضرب = 1 ؛

إذا (الرقم < 0) فإن {

طالما (الرقم < 0) استمر {

المجموع = المجموع + الرقم ؛

الضرب = الضرب * الرقم ؛

الرقم = الرقم - 1 ؛

؛ {

؛ {

اطبع ("المجموع التراكمي:" , المجموع) ؛

اطبع ("المضروب الرياضي:" , الضرب) ؛

{ .

الهدف: إثبات تكامل الثوابت والمتغيرات والشروط وحلقات التكرار في برنامج واحد.

المخرجات المتوقعة: المجموع التراكمي: 15 | المضروب الرياضي: 120

المثال 9: كشف الأخطاء النحوية (Syntax Errors Negative Test)

برنامج خطأ نحوي تجريبي ؛

متغير

س , ص : صحيح ؛

}

س = 10

ص = س + 5 ؛

إذا (س < 5) {

اطبع (س) ؛

{ ؛

. {

الهدف: اختبار متانة المحلل النحوي في كشف نسيان الفاصلة المنقوطة وتعاقب المعاملات ومواصلة الإعراب (Panic Mode).

المخرجات المتوقعة: ظهور رسائل خطأ نحوي تحدد السطر بدقة والتظليل التلقائي.

المثال 10: كشف الأخطاء الدلالية وحماية الثوابت (Semantic Errors Negative Test)

برنامج خطأ دلالي تجريبي ؛

ثابت

الحد_الثابت = 50 ؛

متغير

س : صحيح ؛

}

س = 20 ؛

الحد_الثابت = 100 ؛

غير_معرف = س + 5 ؛

اقرا (الحد_الثابت) ؛

. {

الهدف: اختبار المحلل الدلالي في منع تعديل الثوابت ومنع القراءة إليها وكشف المتغيرات غير المصرح بها بدقة متناهية.

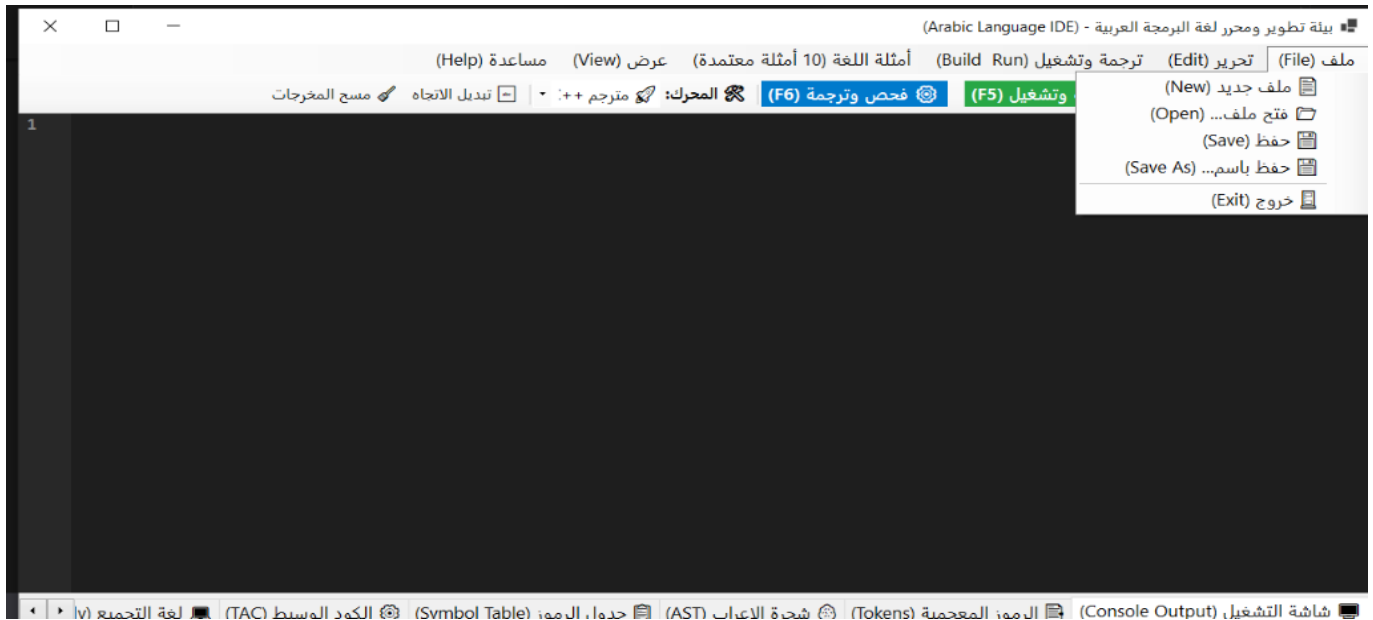
المخرجات المتوقعة: كشف محاولة تعديل الثابت ومنع القراءة إليه وتحديد أسطر الأخطاء.

7. التوثيق التطبيقي المصور ومخرجات تنفيذ المشروع



الشكل (4): واجهة المحرر التفاعلية الرئيسية وتشغيل مخرجات الكونسول (Console Output)

توضح الصورة بيئة التطوير الرسومية المتكاملة لمشروعنا؛ ويظهر شريط القوائم والأدوات مع اختيار مترجم C++20 المستقل، ومحرر الكود المصدري العربي بترميز UTF-8، وفي الأسفل تبويب شاشة الكونسول حيث تم ترجمة برنامج العمليات الحسابية وتشغيله بنجاح وعرض النواتج الفعلية، مع حقل مدخلات تعليمة اقرا وزر تحديث الواجهة.



الشكل (5): القوائم الرئيسية وإدارة الملفات والمشاريع (File Operations)

توضح الصورة القائمة المنسدلة لملف (File) في المحرر، وتوفر الأوامر الأساسية: إنشاء ملف جديد (New)، فتح ملف عربي (Open)، حفظ الملف (Save)، حفظ باسم (Save As)، والخروج من البيئة مع دعم كامل لاختصارات لوحة المفاتيح والترميز العربي.

رقم السطر	النوع	قيمة الرمز (Token Value)
1	..Keyword	برنامج
2	..Ident	فحص_البيانات المنطقية
3	..Ident	:
4	..Keyword	متغير
5	..Ident	ن
6	..Sym	,
7	..Ident	ن
8	..Sym	:
9	..Keyword	صحيح
10	..Ident	:
11	..Ident	حالة_التحقق
12	..Sym	:

الشكل (6): جدول استخراج الرموز المعجمية (Tokens DataGridView)

توضح الصورة جدول الرموز المعجمية الناتج عن مرحلة التحليل المعجمي (Lexical Analysis) في مترجم ++C؛ وتظهر شبكة البيانات مفهرسة بالأعمدة: الرقم التسلسلي، نوع الرمز (Keyword, Identifier, Symbol)، قيمة الرمز المستخرج، ورقم السطر البرمجي بدقة تامة.

```

StatementList (السطر 9)
├── Assign (ن) (السطر 9)
│   └── Variable (ن) (السطر 9)
│       └── Number = 50 (السطر 9)
├── Assign (ص) (السطر 10)
│   └── Variable (ص) (السطر 10)
│       └── Number = 30 (السطر 10)
├── Assign (حالة_التحقق) (السطر 11)
│   └── Variable (حالة_التحقق) (السطر 11)
│       └── BooleanLiteral = صحيح (السطر 11)
└── IfStatement (السطر 14)
    ├── BinaryExpr (8<8) (السطر 14)
    │   ├── BinaryExpr (8<8) (السطر 14)
    │   │   ├── Variable (ن) (السطر 14)
    │   │   └── Variable (ص) (السطر 14)
    │   └── BinaryExpr (=) (السطر 14)
    │       ├── Variable (ص) (السطر 14)
    │       └── Number = 0 (السطر 14)
    └── Variable (حالة_التحقق) (السطر 14)
        └── StatementList (السطر 15)
            └── PrintStatement (السطر 15)
                └── StringLiteral = الشرط المنطقي المركب صحيح بالكامل (السطر 15)
                    └── ElseBlock (السطر 17)
                        └── StatementList (السطر 17)
                            └── PrintStatement (السطر 17)
                                └── StringLiteral = الشرط المنطقي غير محقق (السطر 17)
                                    └── PrintStatement (السطر 20)
                                        └── StringLiteral = نهاية مدخل المنطق (السطر 20)
    
```

الشكل (7): شجرة الإعراب والاشتقاق النحوي الهرمية (AST Parse Tree)

توضح الصورة العرض الشجري التفاعلي لشجرة الإعراب المجردة (AST) الناتجة عن المحلل النحوي في مشروعنا، حيث تظهر التفرعات الهرمية لقوائم الجمل، وجمل الإسناد، والتعبيرات الحسابية والمنطقية، والكتل الشرطية بتدرج لوني وبصري منظم.

اسم الرمز	نوع البيانات	التصنيف	القيمة	سطر التعريف
50	متغير	متغير	-	5
30	متغير	متغير	-	1
حالة التحقق	متغير	متغير	-	5
حالة التحقق	متغير	متغير	-	6

الشكل (8): جدول الرموز وإدارة النطاقات والمراجع (Symbol Table DataGridView)

توضح الصورة جدول الرموز الشامل في بيئة التطوير؛ حيث يوثق كافة المعارف في البرنامج، نوع البيانات (صحيح، منطقي..)، التصنيف (متغير، ثابت)، سطر التعريف، وأسطر الاستخدام التراكمية في البرنامج لضمان التحقق الدلالي التام.

```

// =====
// لتوليد: محصر العمليات المنطقية (TAC) بداية الكود الربط
// =====
50 = 50
30 = 30
حالة التحقق = صحيح
t1 = 50
t2 = 0
t3 = t1 && t2
t4 = t3 && t2
if t4 goto L1
L1:
print "الشرط المنطقي المركب صحيح بالكامل"
goto L3
L2:
print "الشرط المنطقي غير صحيح"
L3:
print "نهاية فحص المنطق"
// نهاية البرنامج
    
```

الشكل (9): الشفرة الوسيطة ثلاثية العناوين (Three-Address Code – TAC)

توضح الصورة كود العناوين الثلاثية المولد فعلياً من مترجم ++C، مع تخصيص المتغيرات المؤقتة (t1, t2..). لتفكيك العمليات الحسابية والمنطقية المركبة، واستخدام تعليمات القفز الشرطي (if condition goto L1) وتعليمات read و print.

شاشة التشغيل (Console Output) الرموز المعجمية (Tokens) شجرة الإعراب (AST) جدول الرموز (Symbol Table) الكود الوسيط (TAC) لغة التجميع (x86 Assembly) NET CIL قائمة الأخطاء

```
<?xml-namespace prefix='asm' namespace='http://www.felixcloutier.com/x86'><asm><?xml-namespace prefix='asm' namespace='http://www.felixcloutier.com/x86'><code><?xml-namespace prefix='asm' namespace='http://www.felixcloutier.com/x86'><code>
```

الشكل (10): شفرة لغة التجميع x86 المولدة من المترجم (x86 Assembly Code)

وتوليد تعليمات المعالج x86 في قسم الشفرة (.code / main PROC) باستخدام سجلات المعالج وسياق الاستدعاء القياسي. (data.)

#	السطر	نوع الخطأ	نص وتفاصيل الخطأ وموقعه في الكود
1	9	خطأ دلالي (Semantic)	خطأ دلالي في السطر 9: إعادة تعريف المتغير 'س'.
2	12	خطأ دلالي (Semantic)	خطأ دلالي في السطر 12: لا يمكن تغيير قيمة الثابت 'الحل_الثاني'.
3	13	خطأ دلالي (Semantic)	خطأ دلالي في السطر 13: محاولة الاستناد إلى متغير غير معرف 'متغير_مجهول'.
4	13	خطأ دلالي (Semantic)	خطأ دلالي في السطر 13: استخدام المعرف غير المعروف 'متغير_مجهول'.

الشكل (11): قائمة تشخيص الأخطاء التفاعلية والتحقق الدلالي (Error Diagnostic List)

توضح الصورة شبكة تشخيص الأخطاء عند فحص كود غير سليم؛ حيث يكشف المحلل النحوي والدلالي الأخطاء اللحظية بدقة (محاولة تعديل الثابت، استخدام متغير غير معلن) مع توفير الانتقال التلقائي الفوري لموضع الخطأ عند النقر عليه.

8. شروط وملفات التسليم النهائي للمشروع

وفقا للتعليمات والضوابط الرسمية لمشروع مقرر بناء المترجمات، تم إعداد الملفات الثلاثة الأساسية للتسليم وتسميتها وفق كود المجموعة المعتمد (GN = GXX):

1. ملف التقرير الأكاديمي الشامل (RPT-GXX.pdf): بصيغة PDF موثق أكاديميا بالكامل وشامل لكافة الجوانب الهندسية.
2. الملف التنفيذي المباشر للمشروع (PREXE-GXX.exe): حزمة تشغيل مباشرة وتثبيت ذاتي تجمع المترجم والمحرر معا.
3. ملف الأكواد والمشاريع المصغرة (PRFL-GXX.rar): أرشيف مضغوط يحوي كافة الأكواد المصدريّة لمترجم ++C ومحرر #C وحزمة الأمثلة العشرة وملفات البناء المباشر بدون أي ملفات مؤقتة غير ضرورية.

9. المراجع والمصادر العلمية المعتمدة (References – APA 7th Edition)

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley.
- [2] Al-Kahsah, K. M. (2025). *Compiler construction course project specifications and Arabic grammar rules (CS 351)*. Faculty of Computer Science and Information Technology, Ibb University.
- [3] Al-Sharfi, A. (2026). *Practical compiler construction and laboratory delivery guidelines*. Faculty of Computer Science and Information Technology, Ibb University.
- [4] Cooper, K. D., & Torczon, L. (2012). *Engineering a compiler* (2nd ed.). Morgan Kaufmann.
- [5] Grune, D., & Jacobs, C. J. H. (2008). *Parsing techniques: A practical guide* (2nd ed.). Springer.
- [6] International Organization for Standardization. (2020). *Programming languages — C++ (ISO/IEC 14882:2020)*. ISO.
- [7] Irvine, K. R. (2014). *Assembly language for x86 processors* (7th ed.). Pearson.
- [8] Loudon, K. C. (1997). *Compiler construction: Principles and practice*. PWS Publishing Company.
- [9] Microsoft Corporation. (2024). *C# language reference, version 13.0 (.NET 9.0)*. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/>
- [10] Scott, M. L. (2015). *Programming language pragmatics* (4th ed.). Morgan Kaufmann.
- [11] Unicode Consortium. (2023). *The Unicode standard, version 15.1.0*. <https://www.unicode.org/>